

模块 A：ReClaim REST API

- 竞赛时长：3 小时
- 技术栈：任意服务端框架 + MySQL

场景

上海浦东国际机场（PVG）委托开发 ReClaim，作为覆盖全机场的统一失物招领服务。失物招领工作人员可以登记捡拾物品并处理来自任意航站楼的旅客申领；旅客可以在线报告遗失物品并跟踪申领进度；系统会为每一份新申领推荐最匹配的库存物品。

工作人员桌面应用和旅客门户由其他团队开发。你需要交付它们背后的 REST API，并严格按照下列规范实现。系统将使用 Bruno 自动化测试套件进行测试；响应结构、HTTP 状态码和业务逻辑必须与预期完全一致。

环境准备

- 导入 MySQL 数据库转储：`dist/database/reclaim-db.sql`。
- 同一数据集还以 JSON 文件形式提供在 `dist/data/json/` 中作为备用。SQL 转储是主要来源，评分使用 MySQL 数据库。
- 下文所有端点路径均相对于 API 基础 URL。基础 URL 可以位于域名根路径或子路径，例如：
 - `http://localhost:8000/api`
 - `https://wsXX-YYYY-module-a.foredu.cn/api`
- 无论 API 挂载在何处都必须正常工作。需要在测试套件环境中填写基础 URL。
- Swagger UI 文档位于 `dist/api-docs/`，在浏览器中打开 `index.html`。
- 所有响应必须为 JSON，任何位置都不得返回 HTML。
- 所有工作人员端点需要通过 `POST /login` 获取的 Bearer 令牌。
- 所有旅客门户端点需要通过 `POST /passenger/login` 获取的独立旅客 Bearer 令牌。
- `GET /claims/track/{referenceCode}` 和 `POST /passenger/register` 为公共端点，无需认证。

使用 Bruno 测试

评分所使用的 Bruno 测试集位于 `dist/api-tests/`，应在开发过程中持续使用。

1. 在 Bruno 中选择 **Open Collection**，打开 `api-tests` 文件夹。
2. 如果 Bruno 询问 JavaScript 沙箱模式，选择 **Developer Mode**，数据库重置步骤需要该模式。
3. 选择 **Local** 环境并填写：
 - `baseUrl`：API 实际地址。
 - `dbName`、`dbUser`、`dbPass`：MySQL 连接信息。
 - 如果不是本地默认连接，填写 `dbHost` 和 `dbPort`。
 - 如果 `mysql` 命令不在 PATH 中，在 `mysqlPath` 填写 MySQL 命令行客户端完整路径。

测试名称包含评分细则编号（A1.1 至 A9.4）及其表述。通过的测试与评分人员执行的检查相同，顺序也与评分表一致。

开发期间建议按以下顺序逐部分测试：

1. **A - reset**：重新导入数据库转储。部分测试会修改数据，所以每次先执行此步骤。
2. **A1 - auth**：登录并保存其他测试文件夹使用的令牌。
3. 运行当前开发部分对应的文件夹，例如 **A3 - items**。

A8 - passenger-portal 会自行执行旅客登录，但仍需先完成数据库重置和工作人员登录，因为部分测试会使用工作人员令牌。

从集合根目录运行可执行全部测试。测试文件夹按顺序运行，开始和结束时都会自动重置数据库，因此整套测试可重复执行。

身份认证

系统包含两套彼此独立的认证机制。

工作人员认证

POST /login

请求：

```
{
  "email": "admin1@reclaim-pvg.cn",
  "password": "admin123"
}
```

成功响应 200：

```
{
  "data": {
    "token": "kJH8s2Lq...60chars",
    "user": {
      "id": 1,
      "name": "...",
      "email": "...",
      "role": "admin"
    }
  }
}
```

凭据无效或账户已停用时返回 401：

```
{ "message": "Invalid credentials" }
```

POST /logout

使当前令牌失效，无请求体。成功响应 200：

```
{ "message": "Logged out successfully" }
```

所有认证端点都必须包含请求头：

```
Authorization: Bearer {token}
```

未认证请求返回 401：

```
{ "message": "Unauthenticated" }
```

旅客认证

旅客通过 POST /passenger/register 或 POST /passenger/login 获取独立令牌。

- 工作人员令牌在 /passenger/* 端点会被拒绝。
- 旅客令牌在工作人员端点会被拒绝。
- 唯一例外是 GET /terminals，它可接受工作人员令牌或旅客令牌。

角色与权限

- **Admin**：可访问所有端点和全部数据。
- **Agent**：只能查看已分配航站楼的数据，可以为这些航站楼登记物品并处理申领。
- Agent 与航站楼是多对多关系，所有查询必须限制在其已分配航站楼范围内。

禁止执行的操作返回 403：

```
{ "message": "Forbidden" }
```

种子数据

航站楼

代码	名称
PVG-T1	Terminal 1
PVG-T2	Terminal 2
PVG-S1	Satellite Hall S1
PVG-S2	Satellite Hall S2

工作人员

邮箱	密码	角色	航站楼
admin1@reclaim-pvg.cn	admin123	Admin	全部
admin2@reclaim-pvg.cn	admin123	Admin	全部
agent1@reclaim-pvg.cn	agent123	Agent	PVG-T1、PVG-S1
agent2@reclaim-pvg.cn	agent123	Agent	PVG-T2、PVG-S2

旅客

邮箱	密码	姓名	国家	说明
passenger1@email.com	passenger123	Li Wei	China	有效账户
passenger2@email.com	passenger123	Maria Santos	Portugal	有效账户
passenger31@email.com	passenger123	Chen Ming	China	已停用账户

物品类别

API 中必须使用以下英文枚举值：

electronics、documents、luggage、clothing、jewellery、keys、other

航站楼端点（3 个）

GET /terminals

返回所有航站楼及统计数量，不分页。Agent 只能看到分配给自己的航站楼。旅客令牌会获得全部航站楼，用于旅客申领表单的航站楼选择器。

```
{
  "data": [
    {
      "id": 1,
      "name": "Terminal 1",
      "code": "PVG-T1",
      "description": "International departures and arrivals, gates A-D",
      "status": "open",
      "items_in_storage_count": 14,
      "open_claims_count": 9,
      "created_at": "2026-...",
      "updated_at": "2026-..."
    }
  ]
}
```

`open_claims_count` 统计状态为 `submitted` 或 `under-review` 的申领数量。

GET `/terminals/{id}`

返回单个航站楼，字段与列表相同。

- 未找到时返回 `404`。
- Agent 访问未分配给自己的航站楼时返回 `403`。

GET `/terminals/{id}/items`

返回指定航站楼的捡拾物品，分页。

查询示例：

```
?status=in-storage
```

捡拾物品端点（5 个）

GET `/items`

列出捡拾物品，每页 15 条，最新的在前。Agent 只能看到其已分配航站楼的物品。

查询示例：

```
?status=in-storage&category=electronics&terminal_id=1&search=iphone
```

`search` 匹配品牌或描述。

```
{
  "data": [
    {
      "id": 1,
      "reference_code": "FI-K3M9P2QX",
      "terminal": {
        "id": 1,
        "name": "Terminal 1",
        "code": "PVG-T1"
      },
      "category": "electronics",
      "brand": "Apple",
      "colour": "black",
      "description": "iPhone 15 Pro, cracked screen protector",
      "found_on": "2026-07-01",
      "found_location": "Gate D68",
      "storage_shelf": "T1-R3-S07",
      "status": "in-storage",
    }
  ]
}
```

```
    "registered_by": {
      "id": 3,
      "name": "Agent One"
    },
    "created_at": "2026-...",
    "updated_at": "2026-..."
  }
],
"links": {},
"meta": {
  "current_page": 1,
  "last_page": 9,
  "per_page": 15,
  "total": 123
}
}
```

`brand`、`colour` 和 `storage_shelf` 允许为 `null`。

GET /items/{id}

返回单个物品。

- 未找到时返回 `404`。
- 物品不属于 Agent 的航站楼时返回 `403`。

POST /items

登记捡拾物品。系统自动生成 `FI-XXXXXXX` 格式的参考编号，初始状态为 `registered`，并自动创建活动日志。

Agent 只能为分配给自己的航站楼登记物品，否则返回 `403`。

```
{
  "terminal_id": 1,
  "category": "electronics",
  "brand": "Apple",
  "colour": "black",
  "description": "iPhone 15 Pro, cracked screen protector",
  "found_on": "2026-07-01",
  "found_location": "Gate D68"
}
```

- `brand` 和 `colour` 可选。
- `found_on` 不得是未来日期。
- 成功时返回 `201` 及创建后的物品。

PUT /items/{id}

部分更新物品详情，所有字段均可选，只发送变化的字段。

```
{
  "brand": "Sony",
  "colour": "navy",
  "storage_shelf": "T1-R2-S04"
}
```

可更新字段：

- category
- brand
- colour
- description
- found_location
- storage_shelf

不能通过此端点修改状态和参考编号。成功时返回 200 及更新后的物品。

PATCH /items/{id}/status

所有字段通过 JSON 请求体发送，此端点没有查询参数。

将物品移入库存时必须提供 storage_shelf，它可以已存在于物品中，也可以与状态一起发送：

```
{
  "status": "in-storage",
  "storage_shelf": "T1-R3-S07"
}
```

保留期结束后处置物品：

```
{"status": "disposed"}
```

允许的转换：

- registered -> in-storage
 - 必须具有 storage_shelf，否则返回 422。
- in-storage -> donated
- in-storage -> disposed
 - 仅当 found_on 距今超过 60 天时允许。
 - 否则返回 422：

```
{"message": "Item is still within the retention period"}
```

`matched` 和 `returned` 不能直接设置，它们由申领 workflow 管理。无效状态转换返回 `422`。每次状态变化都必须写入活动日志。

申领端点（4 个）

GET /claims

列出申领，每页 15 条，最新的在前。Agent 只能看到其已分配航站楼的申领。

查询示例：

```
?status=submitted&category=electronics&terminal_id=1
```

每条申领包含：

- `id`
- `reference_code`
- `passenger`
- `terminal`
- `category`
- `brand`
- `colour`
- `description`
- `lost_on`
- `flight_number`
- `status`
- `matched_item`
- `created_at`
- `updated_at`
- `resolved_at`

`matched_item` 在确认匹配前为 `null`。确认后结构如下：

```
{
  "id": 12,
  "reference_code": "FI-...",
  "storage_shelf": "T1-R3-S07"
}
```

GET /claims/{id}

返回单个申领。

- 未找到时返回 404。
- 申领不属于 Agent 的航站楼时返回 403。

PATCH /claims/{id}/status

请求：

```
{"status": "under-review"}
```

仅允许：

- submitted -> under-review：工作人员开始处理申领。
- matched -> under-review：解除匹配，关联物品恢复为 in-storage，matched_item 变为 null。

其他状态由专用端点管理，在此设置返回 422。无效转换返回 422。成功时返回更新后的申领，并自动创建活动日志。

GET /claims/track/{referenceCode}

公共端点，无需认证。

```
{
  "data": {
    "reference_code": "CL-A7B2C9DE",
    "status": "under-review",
    "category": "electronics",
    "terminal": {
      "name": "Terminal 1",
      "code": "PVG-T1"
    },
    "lost_on": "2026-07-01",
    "created_at": "2026-...",
    "resolved_at": null
  }
}
```

不得暴露旅客个人数据。参考编号未找到时返回 404。

匹配与结案端点（4 个）

典型申领工作流：

1. 旅客提交申领，状态为 submitted。
2. 工作人员通过 PATCH /claims/{id}/status 接手，状态变为 under-review。

- 3. 工作人员调用 `GET /claims/{id}/matches` 获取排序后的建议。
- 4. 工作人员调用 `POST /claims/{id}/match` 确认匹配，申领和物品都变为 `matched`。
- 5. 随后出现三种情况之一：
 - 旅客领取物品：调用 `POST /claims/{id}/resolve`，申领变为 `resolved`，物品变为 `returned`。
 - 匹配错误：通过状态端点将申领退回 `under-review`，清除关联，物品恢复为 `in-storage`。
 - 没有合适物品：对开放申领调用 `POST /claims/{id}/reject`。

GET /claims/{id}/matches

匹配引擎扫描库存物品并按最佳匹配优先返回建议。申领必须处于 `submitted` 或 `under-review`，否则返回 `422`。

第一步：选择候选项

只考虑同时满足以下条件的物品：

- 状态为 `in-storage`。
- 类别与申领相同。

第二步：评分

比较项	分值
品牌相同，不区分大小写，且双方都有值	30
颜色相同，不区分大小写，且双方都有值	25
物品位于申领所指航站楼	20
<code>found_on</code> 与 <code>lost_on</code> 相差 0 至 1 天	25
相差 2 至 3 天	15
相差 4 至 7 天	5
相差 8 天或以上	0

第三步：过滤与排序

- 丢弃得分低于 40 的候选项。
- 按总分从高到低排序。
- 同分时按 `found_on` 从新到旧排序。
- 日期仍相同时按 `id` 从小到大排序。
- 每条结果包含总分和分项得分。

伪代码：

candidates = status 为 "in-storage" 且 category 等于 claim.category 的物品

对每个候选物品：

breakdown.brand = 品牌有效且忽略大小写后相同则 30，否则 0
breakdown.colour = 颜色有效且忽略大小写后相同则 25，否则 0
breakdown.terminal = terminal_id 相同则 20，否则 0

days = found_on 与 lost_on 相差的绝对整数天数
breakdown.date = days <= 1 时为 25
 days <= 3 时为 15
 days <= 7 时为 5
 否则为 0

score = brand + colour + terminal + date

仅保留 score >= 40

按 score 降序、found_on 降序、id 升序排列

返回 { item, score, breakdown }

计算示例：申领类别为 jewellery，品牌 Cartier，颜色 gold，航站楼为 Terminal 1，遗失日期为 7 月 1 日。

候选物品	品牌	颜色	航站楼	日期	总分
Cartier 金色手链，Terminal 1，7 月 1 日捡拾	30	25	20	25	100
Cartier 金色项链，Terminal 2，6 月 29 日捡拾	30	25	0	15	70
Tiffany 银色戒指，Terminal 1，7 月 1 日捡拾	0	0	20	25	45
Pandora 玫瑰金手链，Terminal 2，6 月 21 日捡拾	0	0	0	0	不返回

没有候选项达到 40 分时返回空 data 数组。

种子数据中的申领 CL-F8DA73A7（id 1）就是上述珠宝申领。正确实现必须按顺序返回 100、70、50、45 四个分数，其中 50 分的是未在示例表格中显示的无品牌金色物品。

POST /claims/{id}/match

请求：

```
{"item_id": 12}
```

- 申领必须处于 under-review。
- 物品必须处于 in-storage。
- 物品不一定需要出现在匹配建议中。
- 条件不满足时返回 422。
- 成功后申领和物品都变为 matched，并建立关联。

- 为申领和物品自动创建活动日志。
- 返回 `200` 及包含 `matched_item` 的更新后申领。

POST /claims/{id}/resolve

无请求体。

- 申领必须处于 `matched`，否则返回 `422`。
- 申领变为 `resolved`。
- 自动设置 `resolved_at`。
- 物品变为 `returned`。
- 自动创建活动日志。
- 返回 `200` 及更新后的申领。

POST /claims/{id}/reject

```
{"reason": "No matching item found after 30 days"}
```

`reason` 可选。

- 申领必须处于 `submitted` 或 `under-review`。
- 已匹配申领必须先解除匹配。
- 条件不满足时返回 `422`。
- 申领变为 `rejected`。
- 自动创建活动日志，并将原因写入详情。

仪表板端点（3 个）

GET /dashboard/stats

返回聚合统计数据。Agent 只能看到其已分配航站楼的数据。

```
{
  "data": {
    "total_items": 123,
    "items_in_storage": 67,
    "total_claims": 61,
    "open_claims": 20,
    "today_items": 7,
    "today_claims": 2,
    "items_by_category": {
      "electronics": 18,
      "documents": 23,
      "luggage": 20,
      "clothing": 21,
```

```
    "jewellery": 5,
    "keys": 14,
    "other": 22
  },
  "claims_by_status": {
    "submitted": 10,
    "under-review": 10,
    "matched": 7,
    "resolved": 20,
    "rejected": 8,
    "withdrawn": 6
  }
}
```

`open_claims = submitted + under-review`。

`recent_claims` 包含最近 5 条申领，最新的在前，每条至少包含：

- `id`
- `reference_code`
- `passenger_name`
- `category`
- `status`
- `date`

GET /dashboard/activity

返回最近 20 条工作人员活动日志，最新的在前。

- Agent 只能看到与其航站楼物品和申领相关的日志。
- 每条记录必须关联一个工作人员用户。
- 不显示没有关联用户的旅客自助操作。
- 根据操作类型，`item` 或 `claim` 可以为 `null`。

GET /my-terminals

仅 Agent 可访问。返回分配给当前 Agent 的所有航站楼，并包含：

- 航站楼详情。
- 今日登记物品。
- 开放申领。

Admin 访问时返回 `403`。

旅客门户端点 (9 个)

所有端点使用 `/passenger/*` 前缀。旅客只能访问自己的数据。

POST /passenger/register

公共端点。

```
{
  "first_name": "Chen",
  "last_name": "Jing",
  "email": "chen.jing@email.com",
  "phone": "+8613912345678",
  "address_1": "88 Century Avenue",
  "address_2": null,
  "city": "Shanghai",
  "postcode": "200120",
  "country": "China",
  "password": "mypassword1"
}
```

- `phone` 和 `address_2` 可选。
- 邮箱必须唯一，否则返回 `422`。
- 密码至少 8 个字符。
- 成功时返回 `201`。
- 注册成功后立即登录，响应包含旅客令牌和资料。

POST /passenger/login

```
{
  "email": "passenger1@email.com",
  "password": "passenger123"
}
```

成功响应结构与注册相同。凭据无效或账户已停用时返回 `401`。

POST /passenger/logout

无请求体。成功响应：

```
{"message": "Logged out successfully"}
```

GET /passenger/claims

返回当前旅客自己的申领，每页 15 条，最新的在前。

查询示例：

```
?status=submitted
```

结构与 GET /claims 相同，但不包含 passenger 对象。

GET /passenger/claims/{id}

返回当前旅客自己的单个申领。未找到或不属于当前旅客时返回 404。

POST /passenger/claims

提交申领。系统自动生成 CL-XXXXXXX 格式参考编号，初始状态为 submitted，并创建一条不关联工作人员用户的活动日志。

```
{
  "terminal_id": 1,
  "category": "electronics",
  "brand": "Apple",
  "colour": "black",
  "description": "Black iPhone with red case, lock screen photo of a dog",
  "lost_on": "2026-07-01",
  "flight_number": "MU583"
}
```

- brand、colour 和 flight_number 可选。
- lost_on 不得是未来日期。
- 成功时返回 201 及创建后的申领。

POST /passenger/claims/{id}/withdraw

- 仅限当前旅客自己的申领。
- 无请求体。
- 申领必须处于 submitted 或 under-review，否则返回 422。
- 状态变为 withdrawn。
- 返回 200 及更新后的申领。
- 自动创建不关联工作人员用户的活动日志。

GET /passenger/profile

返回当前旅客资料：

- `id`
- `first_name`
- `last_name`
- `email`
- `phone`
- `address_1`
- `address_2`
- `city`
- `postcode`
- `country`

绝不能暴露密码或 `api_token`。

PUT /passenger/profile

支持部分更新，只发送发生变化的字段。

```
{
  "phone": "+8613800138099",
  "city": "Shanghai"
}
```

可更新字段：

- `first_name`
- `last_name`
- `email`，必须唯一
- `phone`
- `address_1`
- `address_2`
- `city`
- `postcode`
- `country`
- `password`，至少 8 个字符

成功时返回 `200` 及更新后的资料，结构与 `GET /passenger/profile` 相同。

响应结构

普通成功响应：

```
{ "data": {} }
```

`data` 可以是单个对象或数组。

分页响应：

```
{
  "data": [],
  "links": {},
  "meta": {
    "current_page": 1,
    "last_page": 1,
    "per_page": 15,
    "total": 0
  }
}
```

错误响应：

状态码	含义	响应
401	未认证	<code>{ "message": "Unauthenticated" }</code>
403	禁止访问	<code>{ "message": "Forbidden" }</code>
404	未找到	<code>{ "message": "Resource not found" }</code>
422	验证失败	<code>{ "message": "Validation failed", "errors": {...} }</code>
422	违反业务规则	<code>{ "message": "<descriptive message>" }</code>

Bruno 会逐字符断言本文明确给出的英文消息。必须完全照写，任何拼写差异都会导致测试失败。仅写为 `<descriptive message>` 的位置可自行措辞，测试只检查状态码。

通用要求

- 所有响应必须为 JSON。
- 明确给出的英文错误消息必须完全一致。
- 所有端点必须使用规定的 HTTP 状态码：
 - 成功 GET 返回 `200`。
 - `POST /items`、`POST /passenger/claims`、`POST /passenger/register` 成功时返回 `201`。
 - 其他成功请求返回 `200`。
 - 如果框架默认状态码不同，必须覆盖默认值。
- 所有分页端点每页 15 条。

- 所有工作人员端点使用工作人员 Bearer 令牌。
- 所有旅客门户端点使用独立旅客 Bearer 令牌。
- 两套令牌不能互换。
- 注册和公共申领跟踪无需认证。
- Admin 可查看全部数据，Agent 仅限已分配航站楼。
- 旅客端点仅限当前旅客自己的数据。
- 以下操作必须自动生成活动日志：
 - 物品登记。
 - 物品状态变化。
 - 提交申领。
 - 申领状态变化。
 - 匹配。
 - 结案。
 - 拒绝。
 - 撤回。
- 必须强制执行物品和申领状态转换，无效转换返回 422。
- 匹配建议必须严格使用规定的分值、过滤和排序。
- 启用 CORS 响应头。
- 密码必须以哈希形式存储。
- 共 30 个端点。